数据与算法

# 09 数值计算基本概念

北京科技大学
计算机与通信工程学院
皇甫伟/杜　冰

# Previously …

- **基本概念**
  - 数据、算法、数学建模、时间/空间复杂度、关系、抽象数据类型
- **非数值（有限离散）数据与算法**
  - 线性表：顺序表、链表、循环链表；插入、删除
  - 线性表特例：栈、队列、字符串；递归及消除
  - 非线性结构之一：树、二叉树、二叉搜索树(BST)、AVL树，堆；树遍历、Huffman树、BST的查找/插入/删除、AVL树的构造
  - 非线性结构之二：图(邻接矩阵、邻接表)；图遍历(BFS、DFS)，最小生成树，最短路径
  - 非数值数据应用：查找(线性、折半、散列、索引)、排序（冒泡、插入、选择、快速、归并、堆排序、计数）

# 数值计算

- **数值（无限连续）数据与算法**
  - 数值基本概念：实数、浮点数；误差分析、舍入、抵消、有效数字、条件数
  - 方程求解：恰定线性方程组、非线性方程；高斯消元、选主元素、LU分解、乔利斯基分解、雅可比迭代、高斯-赛德尔迭代、二分法、牛顿法、割线法、反插法
  - 拟合与插值：过定线性方程组、范德蒙矩阵；正规矩阵法、Householder变换、拉格朗日插值、牛顿插值、样条插值
  - 函数优化：目标函数、约束条件；牛顿法、障碍法

# More …

### Arrays [ edit ]

- Array
- Bit array
- Bit field
- Bitboard
- Bitmap
- Circular buffer
- Control table
- Image
- Dope vector
- Dynamic array
- Gap buffer
- Hashed array tree
- Heightmap
- Lookup table
- Matrix
- Parallel array
- Sorted array
- Sparse array
- Sparse matrix
- Iliffe vector
- Variable-length arra

### Lists [ edit ]

- Doubly linked list
- Array list
- Linked list
- Self-organizing list
- Skip list
- Unrolled linked list
- VList
- Xor linked list
- Zipper
- Doubly connected edge lis
- Difference list
- Free list

### B-trees [ edit ]

- B-tree
- B+ tree
- B*-tree
- B sharp tree
- Dancing tree
- 2-3 tree
- 2-3-4 tree
- Queap
- Fusion tree
- Bx-tree
- AList

### Binary trees [ edit ]

- AA tree
- AVL tree
- Binary search tree
- Binary tree
- Cartesian tree
- Order statistic tree
- Pagoda
- Randomized binary search tree
- Red-black tree
- Rope
- Scapegoat tree
- Self-balancing binary search tree
- Splay tree
- T-tree
- Tango tree
- Threaded binary tree
- Top tree
- Treap
- WAVL tree
- Weight-balanced tree

### Graphs [ edit ]

- Graph
- Adjacency list
- Adjacency matrix
- Graph-structured stack
- Scene graph
- Binary decision diagram
- Zero-suppressed decision diagram
- And-inverter graph
- Directed graph
- Directed acyclic graph
- Propositional directed acyclic graph
- Multigraph
- Hypergraph

### Heaps [ edit ]

- Heap
- Binary heap
- Weak heap
- Binomial heap
- Fibonacci heap
  - AF-heap
- Leonardo Heap
- 2-3 heap
- Soft heap
- Pairing heap
- Leftist heap
- Treap
- Beap
- Skew heap
- Ternary heap
- D-ary heap
- Brodal queue

### Hashes [ edit ]

- Bloom filter
- Count-Min sketch
- Distributed hash table
- Double Hashing
- Dynamic perfect hash table
- Hash array mapped trie
- Hash list
- Hash table
- Hash tree
- Hash trie
- Koorde
- Prefix hash tree
- Rolling hash
- MinHash
- Quotient filter
- Ctrie

### Space-partitioning trees [ edit ]

These are data structures used for spa

- Segment tree
- Interval tree
- Range tree
- Bin
- Kd-tree
- Implicit kd-tree
- Min/max kd-tree
- Adaptive k-d tree
- Quadtree
- Octree
- Linear octree
- Z-order
- UB-tree
- R-tree
- R+ tree
- R* tree
- Hilbert R-tree
- X-tree
- Metric tree
- Cover tree
- M-tree
- VP-tree
- BK-tree
- Bounding interval hierarchy
- BSP tree
- Rapidly exploring random tree

### Multiway trees [ edit ]

- Ternary tree
- K-ary tree
- And–or tree
- (a,b)-tree
- Link/cut tree
- SPQR-tree
- Spaghetti stack
- Disjoint-set data structure
- Fusion tree
- Enfilade
- Exponential tree
- Fenwick tree
- Van Emde Boas tree
- Rose tree

https://en.wikipedia.org/wiki/List_of_data_structures

# 本节内容

- 数值计算问题
- 近似与误差
- 计算机运算

# 数值计算问题

- **非数值计算问题**
  - 查找、排序、模式匹配、最短路径、最小生成树 …
  - 问题变量通常是离散的；一定规模的问题解空间通常是有限的
  - 精确解如存在，可能在有限步内达到；求解过程通常是稳定的，对输入的依赖性较小
  - 算法的性能通常用时间/空间复杂度衡量
- **数值计算问题**
  - 方程求解、插值、拟合、最优化 …
  - 问题变量和解空间通常是连续的
  - 一般无法通过有限步运算得到精确解，仅能满足于符合精度要求的解(可能有无限多个)
  - 求解过程的稳定程度，取决于其对输入的依赖性
  - 算法的性能通常用收敛性、稳定性以及解的精确度衡量

# 数值计算问题

- **算法不仅仅决定效率，还可能决定结果精确程度**

$$\ln 1.5 = 0.4054651\ldots$$

算法1： $\ln(1+x) \approx f_1(x) = x - \frac{1}{2}x^2 + \frac{1}{3}x^3 - \frac{1}{4}x^4$

$$f_1(x = 0.5) = 0.4010416\ldots$$

算法2： $\ln\left(\frac{1+x}{1-x}\right) \approx f_2(x) = 2x + \frac{2}{3}x^3 + \frac{2}{5}x^5 + \frac{2}{7}x^7$

$$f_2(x = 0.2) = 0.4054649\ldots$$

# 数值计算问题

- **非数值计算问题的特性:** $P$**问题、**$NP$**问题、**$NPC$**问题、**$NP-hard$**问题**
- **数值计算问题的特性**
  - **适定的(well-posed)/不适定的(ill-posed)：**
    - **适定：**数值计算问题的**解存在、唯一、且连续**，依赖于问题的参数
      - 适定问题有机会用稳定算法求解
      - 适定问题中参数微小变动，不会导致解的急剧的不均匀的变化
    - **不适定：**不唯一存在，求解需补充额外假设，称为**规范化/正则(regularization)**
  - **病态问题(ill-conditioned)：**
    - 参数的微小扰动，解产生高度敏感的响应(虽连续)

# 数值计算问题

- 数值计算问题的特性举例
  - 参数不同，数值问题的特性可能发生改变：

求解线性方程组 $\begin{cases} x+y=1 \\ ax+by=1 \end{cases}$

  - $a=b=1$时，方程解不唯一，问题不适定
  - $a=b\neq 1$时，方程解不存在，问题不适定
  - $a\neq b$时，方程解存在唯一，且是参数的连续函数：

$$x=\frac{b-1}{b-a} \qquad y=\frac{a-1}{a-b}$$

  - 当$a\approx b$时，方程解对参数敏感，问题病态：

$$a=2, b=1.999 \quad \rightarrow \quad x=-999 \quad y=+1000$$

$$a=2, b=2.001 \quad \rightarrow \quad x=+1001 \quad y=-1000$$

# 数值计算问题

- **数值计算问题的求解**
  - 适定/不适定，良态/病态，均是问题的的属性，与选择的算法无关
  - 适定的良态问题，算法得到结果的敏感性也不应超过问题本身 (“do not harm”)
  - 数值计算的输入与计算过程都不可避免存在近似：
    - 数学模型简化；测量的精度受限；前级的计算结果存在近似
    - 数值的截断和离散化的近似；数值表示精度有限造成近似

      例：用球表面积公式$A = 4\pi r^2$计算地球表面面积

      - 地球看成球体，是一种理想化的模型近似
      - 半径$r \approx 6370km$存在测量上的近似
      - 圆周率$\pi$的值只能截取到某一位
      - 计算过程存在数据的舍入

# 本节内容

- 数值计算问题
- 近似与误差
- 计算机运算

# 近似与误差

- **误差(error)**
  - 测量或计算结果与真值之间的偏差
  - 数值计算中各种近似综合作用反映为计算结果的误差
  - 计算过程中误差可能传递、积累或者是对消
  - 绝对误差 = 近似值- 真值
    - 精确度：近似某个量时有效数字的位数
    - 精度：表示数的数字个数（不一定都是有效数字）
  - 相对误差 = 绝对误差/真值 (真值 ≠ 0)
  - 近似值 = 真值 × (1 + 相对误差)
  - 真值一般未知，通常只能估计误差范围，相对误差只能相对于近似值

$$\text{误差}\begin{cases}\text{数据误差}\\ \text{计算误差}\begin{cases}\text{截断误差}\\ \text{舍入误差}\end{cases}\end{cases}$$

# 近似与误差

- **有效数字(significant digits)**
  - 除了误差之外，实际中也常用有效数字的位数来反映近似数的准确程度

绝对误差

  - 用$x^*$来近似$x$，若$|\mathbf{x}-\boldsymbol{x}^*| \le \frac{1}{2}\times 10^{-k}$，则称该近似精确到**小数点之后第k位**

    称从小数点之后第$k$位起直到最左边的非零数字之间的一切数字为有效数字，有效数字的位数称为**有效位数**

$$\pi = 3.1415926535897932\ldots$$

$\pi^* = 3.14$ $\quad |\pi-\pi^*| \approx 0.001526 \le 0.5\times 10^{-2}$ $\quad$ 有效位数=3

$\pi^* = 3.1416$ $\quad |\pi-\pi^*| \approx 0.00000735 \le 0.5\times 10^{-4}$ $\quad$ 有效位数=5

$\pi^* = 3.1415$ $\quad |\pi-\pi^*| \approx 0.0000927 \le 0.5\times 10^{-3}$ $\quad$ 有效位数=4

# 近似与误差

- **有效数字 vs 相对误差**
  - 若近似值$x^*$的有效位数为$n$，则其相对误差满足：

$$|\text{相对误差}| \le \frac{1}{2x_1} \times 10^{-(n-1)}$$

，其中$x_1$为最左侧的一位有效数字

---

$$\pi = 3.1415926535897932\ldots$$

$$\pi^* = 3.14 \qquad \frac{|\pi - \pi^*|}{\pi} \approx 0.000486 \le 0.00167 \approx \frac{1}{2 \times 3} \times 10^{-(3-1)}$$

$$\pi^* = 3.1416 \qquad \frac{|\pi - \pi^*|}{\pi} \approx 0.00000234 \le 0.0000167 \approx \frac{1}{2 \times 3} \times 10^{-(5-1)}$$

$$\pi^* = 3.1415 \qquad \frac{|\pi - \pi^*|}{\pi} \approx 0.0000295 \le 0.000167 \approx \frac{1}{2 \times 3} \times 10^{-(4-1)}$$

# 近似与误差

近似$\sqrt{20}$并**保证**其相对误差不超过0.1%，需要至少保留多少位有效数字？

假设有效位数至少为$n$，则应该有 $\frac{1}{2x_1} \times 10^{-(n-1)} \leq 0.1\%$

易知 $4 \leq \sqrt{20} \leq 5$，因此有 $x_1 = 4$；

于是有 $n \geq log_{10}\left(\frac{1}{2\times4\times0.1\%}\right) + 1 \approx 3.097$

因此，有效位数至少应为4.

$\sqrt{20} = 4.472$ 实际相对误差 ~ 0.003%

# 近似与误差

- 有效数字 $vs$ 相对误差
  - 若近似值$\hat{x}^*$的相对误差满足：

$$|\text{相对误差}| \leq \frac{1}{2(x_1+1)} \times 10^{-(n-1)}$$

则其至少有$n$位有效数字

# 近似与误差

- 数据误差 $vs$ 计算误差
  - $x$表示输入数据真值；$f(x)$表示真实的结果
  - $\hat{x}$表示有误差的输入数据；$\hat{f}(\hat{x})$表示计算的近似值

$$误差 = \hat{f}(\hat{x}) - f(x) = \boxed{\hat{f}(\hat{x}) - f(\hat{x})} + \boxed{f(\hat{x}) - f(x)}$$

计算误差：由算法决定

数据误差：误差的传播

---

- $x = \pi/8$，$f(x) = \sin(x)$ ；$\hat{x} = 3/8$，$\hat{f}(x) = x$；

$$误差 = \hat{f}(\hat{x}) - f(x) \approx 0.3750 - 0.3827 = -0.0077$$

$$计算误差 = \hat{f}(\hat{x}) - f(\hat{x}) \approx \hat{x} - \sin(\hat{x}) = 0.3750 - 0.3663 = 0.0087$$

$$数据误差 = f(\hat{x}) - f(x) \approx \sin(\hat{x}) - sin\left(\frac{\pi}{8}\right) = 0.3663 - 0.3827 = -0.0164$$

# 近似与误差

- 截断误差与舍入误差
  - 计算误差 = 截断误差 + 舍入误差
  - **截断误差(truncation error)**：对于实际输入，真实结果与用给定算法"精确运算"得到结果之间的偏差；由无穷级数的截断、差分代替导数或迭代终止等造成
  - **舍入误差(rounding error)**：用给定算法"精确运算"得到结果，与用同样算法经"**有限精度**"运算(舍入)得到的结果之间的偏差；通常由数的不精确表示造成

---

$x = \pi/8$，$f(x) = \sin(x)$ ；$\hat{x} = 3/8$，$\hat{f}(x) = x$

输入数据3/8的真实结果：$\sin(3/8)$

输入数据3/8用给定算法的精确运算结果： $3/8$

截断误差$= 3/8 - \sin(3/8) \approx 0.0087$

对无限级数$\sin(x) = x - {x^3}/{3!} + {x^5}/{5!} - {x^7}/{7!} + \cdots$ 截断造成

# 近似与误差

- 舍入误差与截断误差之间的相互制约

用前向差分近似微分：$f'(x) \approx \dfrac{f(x+h)-f(x)}{h}$

泰勒展开：　$f(x+h) = f(x) + f'(x)h + f''(\theta)h^2/2, \theta \in [x, x+h]$

**截断误差**的估计为$Mh/2$，其中$M$为求值点附近$|f''(t)|$的上界

**舍入误差**的估计为$2\varepsilon/h$，$\varepsilon$为$f(x)$计算结果精确表示的舍入误差

总的**计算误差**的估计为$Mh/2+ 2\varepsilon/h$：

- 截断误差随$h$的减少而减少；舍入误差随$h$的减少而增加

总**计算误差**的在$h^* = 2\sqrt{\varepsilon/M}$时取最小值

# 近似与误差

- 舍入误差与截断误差之间的相互制约

用三中点差分近似微分：$f'(x) \approx \dfrac{f(x+h)-f(x-h)}{2h}$

泰勒展开：$f(x+h) = f(x) + f'(x)h + \dfrac{f''(\theta)h^2}{2} + \dfrac{f'''(\theta)h^3}{6}$

$$f(x-h) = f(x) + f'(x)h + \frac{f''(\theta)h^2}{2} - \frac{f'''(\theta)h^3}{6}$$

<u>**截断误差**</u>的估计为$Th^2/2$，其中$T$求值点附近$|f'''(t)|$的上界

<u>**舍入误差**</u>的估计为$\varepsilon/h$，其中ε为$f(x)$计算结果精确表示的舍入误差

总<u>**计算误差**</u>的估计为$Th^2/2+\varepsilon/h$，在$h^* = \sqrt[3]{\varepsilon/T}$时取最小值

# 近似与误差

$f(x) = \sin(x)$ 计算$f'(0.8)$，假设$\varepsilon \approx 0.000005$

前向差分 $h^* = 2\sqrt{\varepsilon/M} \approx 0.0049$

| $h$ | 0.0002 | **0.0020** | 0.0200 | 0.2000 |
|---|---|---|---|---|
| $f'(0.8)$ | 0.70000 | **0.69500** | 0.68950 | 0.62055 |
| 误差 $\lvert E\rvert$ | 0.00329 | **0.00171** | 0.00721 | 0.07616 |

三点差分 $h^* = \sqrt[3]{3\,\varepsilon/T} \approx 0.026$

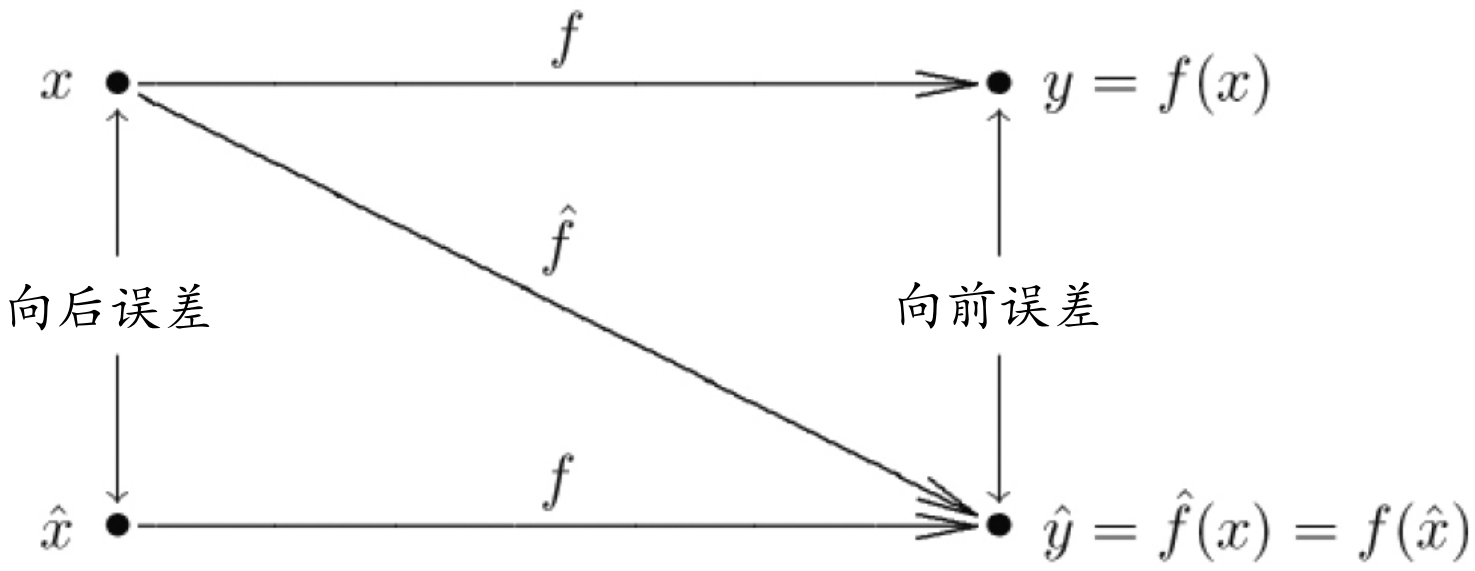
| $h$ | 0.0002 | 0.0020 | **0.0200** | 0.2000 |
|---|---|---|---|---|
| $f'(0.8)$ | 0.70000 | 0.69750 | **0.69675** | 0.69208 |
| 误差 $\lvert E\rvert$ | 0.00329 | 0.00079 | **0.00004** | 0.00463 |

# 近似与误差

- 通常的计算中，截断误差和舍入误差都存在
- 有限次求解的纯代数问题中，**舍入误差主导**
- 涉及积分、导数、非线性等无限求解的问题，**截断误差主导**
- 精确分析不同类型误差，往往难以做到且没有必要，通常将误差合并到一起分析

# 近似与误差

- 向前误差与向后误差
  - 向前误差(forward error)：计算值与真值之间的偏差；通常比较难以估计
  - 向后误差(backward error)：输入数据有多大误差才能产生最终的计算值
    - 若问题的近似解是其”邻近”问题的精确解(向后误差小)，则认为近似是好的



# 近似与误差

余弦函数$y = f(x) = \cos(x)$在$x = 1$处的近似值

余弦函数的无穷级数表示：$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$

级数截取：$\hat{y} = \hat{f}(x) = 1 - \frac{x^2}{2!} = 0.5$

- 向前误差（$y$）
  - 绝对误差$\Delta y = \hat{y} - y = 1 - \frac{x^2}{2!} - \cos(x) \approx -0.0403$
  - 相对误差$\Delta y / y \approx -7.5\%$
- 向后误差 $\hat{x} = \arccos(\hat{y}) = \arccos(0.5) \approx 1.0472$
  - 绝对误差$\Delta x = \hat{x} - x \approx 0.0472$
  - 相对误差$\Delta x / x \approx 4.7\%$

# 近似与误差

- **敏感性与病态性**
  - 某些数值问题本身对数据的扰动高度敏感，即便求解过程”精确”，输入数据的误差也可能被”放大”
  - **良态/不敏感:** 输入的相对偏差对结果影响较适中
  - **病态/敏感:** 输入的相对偏差对结果影响很大
  - **条件数(condition number):** 解的相对变化与输入数据相对变化的比值
    - 病态：条件数$\gg 1$
    - 良态：条件数$< 1$ or 条件数$\approx 1$

$$\text{条件数} = \frac{|\text{相对向前误差}(y)|}{|\text{相对向后误差}(x)|}$$

  - 条件数可以理解为相对向后误差的”**放大因子**”

$$|\text{相对向前误差}(y)| = \text{条件数} \times |\text{相对向后误差}(x)|$$

# 近似与误差

- **敏感性与病态性**
  - 条件数随输入数据的变化而改变，难以求出具体的数值，只粗略估计，或给出上限以估计向前误差的最差情况：

$$|\text{相对向前误差}(y)| \leq \text{条件数上限} \times |\text{相对向后误差}(x)|$$

  - 对于可微函数求值问题 $f: \mathbb{R} \to \mathbb{R}$
    - 绝对向前误差$(y)$：$f(x+\Delta x) - f(x) \approx f'(x)\Delta x$
    - 相对向前误差$(y)$：$\approx f'(x)\Delta x / f(x)$
    - 条件数：$\approx \left| \dfrac{f'(x)\Delta x / f(x)}{\Delta x / x} \right| = \left| \dfrac{x f'(x)}{f(x)} \right|$
  - 反问题的条件数为原问题的倒数；若函数输入或者输出为0，也可以用绝对误差定义条件数(绝对条件数)

# 近似与误差

- 敏感性与病态性

$$f(x) = x^a (x \neq 0) \text{ 的条件数}$$

$$cond(f) \approx \left| \frac{xf'(x)}{f(x)} \right| = \left| \frac{x \cdot ax^{a-1}}{x^a} \right| = |a|$$

$f(x) = \sqrt{x},\ f(x) = \sqrt[3]{x}$：良态

$f(x) = x^{100}$：严重病态

# 近似与误差

$$\text{正切函数} f(x) = tan(x)$$

$$\text{条件数} \approx \left|\frac{xf'(x)}{f(x)}\right| = \left|x\left(\frac{1}{\tan(x)} + \tan(x)\right)\right|$$

- 当$x$接近$\pi/2$的整数倍时，条件数接近无穷大，函数高度敏感：
  - $\tan(1.57079) \approx 1.58058 \times 10^5$
  - $\tan(1.57078) \approx 6.12490 \times 10^4$
  - 输出变化超过是输入变化的二十万倍！
- 相应的反函数$\arctan(x)$在该点极不敏感

# 近似与误差

- **稳定性与精确度**
  - 算法是**稳定的**：如果计算结果对由计算过程所产生的近似扰动相对不敏感，
  - 精确度：问题的计算解和真实解之间的接近程度
    - 由算法的稳定性和问题的敏感性共同决定
  - **对良态问题采用稳定的算法才会得到精确的解**

# 近似与误差

$$I_n = \int_0^1 \frac{x^n}{x+5}dx = \int_0^1 \frac{x^{n-1}(x+5-5)}{x+5}dx$$
$$= \int_0^1 x^{n-1}dx - 5\int_0^1 \frac{x^{n-1}}{x+5}dx = \frac{1}{n} - 5I_{n-1}$$

计算定积分族 $I_n = \int_0^1 \frac{x^n}{x+5}dx$，$n = 0,1,2,\dots,20$

容易得到 $I_n = \frac{1}{n} - 5I_{n-1}$，$n = 1,2,\dots,20$

同时 $I_0 = \int_0^1 \frac{1}{x+5}dx = \int_5^6 \frac{1}{t}dt = \ln 6/5 \approx 0.182322$

使用C语言，双精度浮点数，用迭代算法计算：

| $n$ | $I_n$ | $n$ | $I_n$ |
|---|---|---|---|
| 0 | $1.82322 \times 10^{-1}$ | 11 | $-2.16268 \times 10^{1}$ |
| 1 | $8.83900 \times 10^{-2}$ | 12 | $1.08218 \times 10^{2}$ |
| 2 | $5.80500 \times 10^{-2}$ | 13 | $-5.41011 \times 10^{2}$ |
| 3 | $4.30833 \times 10^{-2}$ | 14 | $2.70513 \times 10^{3}$ |
| 4 | $3.45833 \times 10^{-2}$ | 15 | $-1.35256 \times 10^{4}$ |
| 5 | $2.70833 \times 10^{-2}$ | 16 | $6.76279 \times 10^{4}$ |
| 6 | $3.12500 \times 10^{-2}$ | 17 | $-3.38139 \times 10^{5}$ |
| 7 | $-1.33929 \times 10^{-2}$ | 18 | $1.69070 \times 10^{6}$ |
| 8 | $1.91964 \times 10^{-1}$ | 19 | $-8.45348 \times 10^{6}$ |
| 9 | $-8.48710 \times 10^{-1}$ | 20 | $4.22674 \times 10^{7}$ |
| 10 | $4.34335 \times 10^{0}$ | | |

??? $I_{n-1} > I_n > 0$ !

# 近似与误差

$I_n = \frac{1}{n} - 5I_{n-1}$， $I_{n-1}$的误差在计算$I_n$时被放大到5倍， $I_0$的舍入误差在计算到$I_{20}$时被放大了**$5^{20}$倍**！！！

利用反向递推公式：$I_{n-1} = \frac{1}{5n} - \frac{1}{5}I_n$，$n = 30, \dots, 21, 20, \dots 1$

任意取$I_{30} \approx 1$，存在很大的误差！

| $n$ | $I_n$ | $n$ | $I_n$ |
|---|---|---|---|
| 20 | $7.99762 \times 10^{-3}$ | 9 | $1.69265 \times 10^{-2}$ |
| 19 | $8.40048 \times 10^{-3}$ | 8 | $1.88369 \times 10^{-2}$ |
| 18 | $8.84622 \times 10^{-3}$ | 7 | $2.12326 \times 10^{-2}$ |
| 17 | $9.34187 \times 10^{-3}$ | 6 | $2.43249 \times 10^{-2}$ |
| 16 | $9.89633 \times 10^{-3}$ | 5 | $2.84684 \times 10^{-2}$ |
| 15 | $1.05207 \times 10^{-2}$ | 4 | $3.43063 \times 10^{-2}$ |
| 14 | $1.12292 \times 10^{-2}$ | 3 | $4.31387 \times 10^{-2}$ |
| 13 | $1.20399 \times 10^{-2}$ | 2 | $5.80389 \times 10^{-2}$ |
| 12 | $1.29766 \times 10^{-2}$ | 1 | $8.83922 \times 10^{-2}$ |
| 11 | $1.40713 \times 10^{-2}$ | 0 | $\mathbf{1.82322 \times 10^{-1}}$ |
| 10 | $1.53676 \times 10^{-2}$ | | |

$I_{20}$的误差$= I_{30}$的误差的$\frac{1}{5^{10}} < 1.1 \times 10^{-7}$

6位有效数字均可靠！

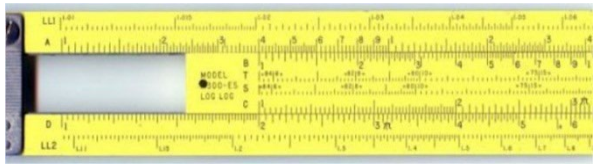
# 本节内容

- 数值计算问题
- 近似与误差
- 计算机运算

# 计算机运算

- **数值运算与计算机**
  - 计数以及对数的运算被认为是数学的主要起源
  - 早期的计算主要依靠人工以及简单的机械工具辅助
  - 科学工程领域大量的实际计算需求推动了计算机的发明
  - 二战期间美国对导弹弹道的研究需要求解大量复杂的非线性方程组，200人2个月才能完成每天需求的1/6
  - Mauchly与1942年提出了电子计算机的初始设想“高速电子管计算装置的使用”，并得到军方项目支持
  - Von Neumann参与该项目并于1945年提出了“存储程序通用电子计算机方案”
  - 1946年2月，第一台计算机$ENIAC$诞生，运算速度不过几千次/秒，但开启了计算机数值运算的时代

计算尺



# 计算机运算

- **浮点数**
  - 计算机广泛应用于数值计算，其中一类常见且难以避免的近似即是实数的表示
  - 基本思想：由适当大小的数乘10的适当次幂表示实数
  - 一个浮点数系统由四个整数表示：
    $\beta$ 基数或底数
    $p$ 精度
    $[L, U]$ 指数$E$范围
  - 任意一个浮点数$x$都具有形式：
    $$x = \pm\left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}}\right)\beta^E$$
  - 其中整数$d_i$满足：$0 \le d_i \le \beta - 1$，$i = 0,1,\ldots,p-1$
  - 括号中的部分用一串$p$位$\beta$进制数$d_0 d_1 \ldots d_{p-1}$表示，称为尾数或者有效数字
  - 整数$E$满足：$L \le E \le U$，称为指数或者特征

$x = (-1)^s(1+f)2^{c-1023}$ $\beta = 2$ $p = f$的位数 $+ 1$

符号 $f$尾数：52 C：指数（如11位，$2^{11}$）

# 计算机运算

- **浮点数**
  - 计算机中符号、指数、尾数均存储于定长的单独域中
  - 零可以用尾数和指数都为零的浮点数表示
  - IEEE的单精度(32)与双精度(64)二进制浮点数应用最为广泛

| 系 统 | $\beta$ | $p$ | $L$ | $U$ |
|---|---|---|---|---|
| IEEE单精度 | 2 | 24 | -126 | 127 |
| IEEE双精度 | 2 | 53 | -1022 | 1023 |
| Cray系统 | 2 | 48 | -16383 | 16384 |
| HP计算器 | 10 | 12 | -499 | 499 |
| IBM主机 | 16 | 6 | -64 | 63 |

8: 0-255

11: 0-2047

# 计算机运算

- **正规化**
  - 正规化的浮点系统中，浮点数(除0外)的尾数$m$的首位数字$d_0 \neq 0$，且$1 \leq m < \beta$
  - 正规化表示下，数的表示方法唯一
    - 首位非零，精度最大化
    - 二进制系统中首位总是1，可以不存储
  - 正规化浮点数系统中，总的可能的浮点数个数为：
    - 符号2种选择：$\pm$
    - 尾数：首位有$\beta - 1$种选择，其余$p - 1$位有$\beta$种选择
    - 指数：$U - L + 1$种
    - 单独加0

$$\boldsymbol{2(\beta - 1)\beta^{p-1}(U - L + 1) + 1}$$

$$x = \pm\left(d_0 + \frac{d_1}{\beta} + \frac{d_2}{\beta^2} + \cdots + \frac{d_{p-1}}{\beta^{p-1}}\right)\beta^E$$

  - 正规化浮点数最小正数(下溢限)：$UFL = \beta^L$
  - 正规化浮点数最大正数(上溢限)：$OFL = \beta^{U+1}(1 - \beta^{-p})$

# 计算机运算

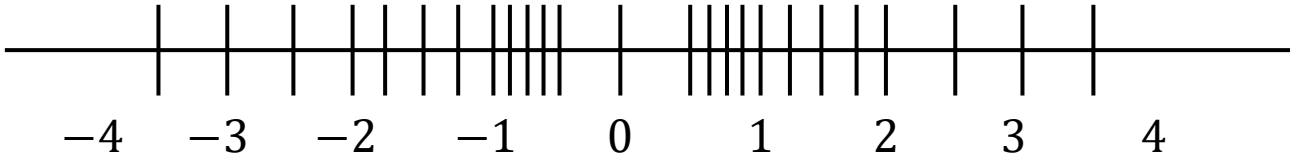
- **正规化**
  - 超过OFL，或是小于UFL的数均不能表示
  - 在表示范围内，浮点数并非均匀分布

例：某正规化浮点数系统中 $\beta = 2, p = 3, L = -1, U = 1$

总的可能的浮点数个数：$2(\beta - 1)\beta^{p-1}(U - L + 1) + 1 = 25$

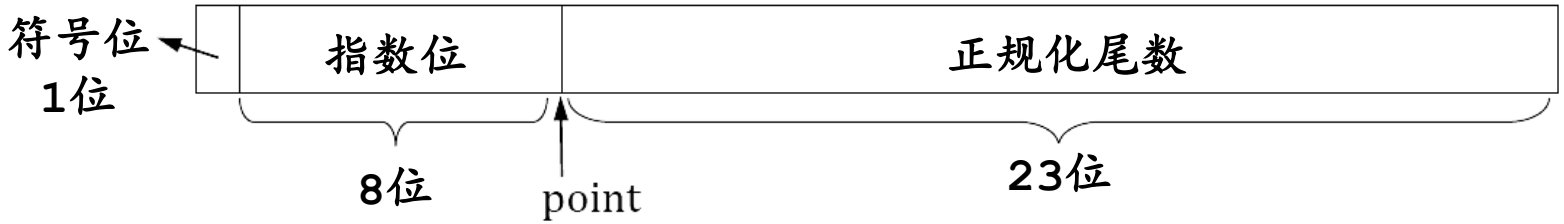
下溢限 $UFL = \beta^{L} = (0.5)_{10}$

上溢限 $OFL = \beta^{U+1}(1 - \beta^{-p}) = (3.5)_{10}$

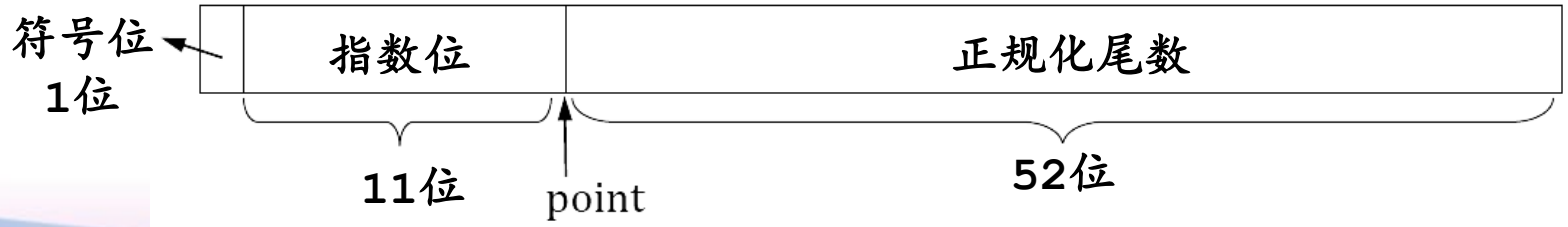


# 计算机运算

- **IEEE标准浮点数系统**
  - 单精度浮点数(float)：32位，符号位1位，尾数24位（正规化表示，首位为1省略），指数8位




  - 双精度浮点数(double)：64位，符号位1位，尾数53位（正规化表示，首位为1省略），指数11位 $x = (-1)^s(1+f)2^{c-1023}$



# 计算机运算

- **IEEE标准浮点数系统**
  - 双精度浮点数(double)： $2(\beta-1)\beta^{p-1}(U-L+1)+1$

    $\beta=2, p=53, L=-1022, U=1023$
  - 总共可以表达约$1.84\times10^{19}$ 个不同的机器数.

例：求IEEE双精度浮点数系统中下述机器数的十进制取值

$$x = 0\ 10000000001\ 1001000100000001000000000000000000000000000000000000$$

指数部分： $c = 2^{10} + 2^{0} = 1025$

尾数部分： $f = \frac{1}{2} + \frac{1}{2^4} + \frac{1}{2^8} + \frac{1}{2^{16}} = 0.5664215087890625$

$$x = (-1)^s(1+f)2^{c-1023} = 2^{1025-1023} * (1+f) = 6.26568603515625$$

# 计算机运算

- **舍入(rounding)**
  - **机器数：**在某浮点系统中精确表示的实数
  - 若一个实数不能精确表示，则选择某"邻近"的浮点数加以近似表示，该选择过程称为**舍入**；舍入过程引入的误差即为**舍入误差**
  - 两个常用的舍入原则：
    - 截断：将$x$的$\beta$基底展开的第$p-1$位之后截去，也称**向零舍入**
    - 最近舍入：取与$x$最接近的浮点数，相等时选取**最末存储位为偶数的浮点数**
      - 较为精确但代价高，IEEE标准系统采用

| 实数 | 截断 | 最近舍入 | 实数 | 截断 | 最近舍入 |
|---|---|---|---|---|---|
| 1.649 | 1.6 | 1.6 | 1.749 | 1.7 | 1.7 |
| 1.650 | 1.6 | 1.6 | 1.750 | 1.7 | 1.8 |
| 1.651 | 1.6 | 1.7 | 1.751 | 1.7 | 1.8 |
| 1.699 | 1.6 | 1.7 | 1.799 | 1.7 | 1.8 |

# 计算机运算

- **舍入(rounding)**
  - 最近舍入：取与$x$最接近的浮点数，相等时选取最末存储位为偶数的浮点数

$$x_+ = 0\ \ 10000000001\ \ 1001000100000001000000000000000000000000000000000001$$
$$x = 0\ \ 10000000001\ \ 1001000100000001000000000000000000000000000000000000$$
$$x_- = 0\ \ 10000000001\ \ 1001000100000000111111111111111111111111111111111111$$

区间$[(x+x_-)/2, (x+x_+)/2]$内的实数都舍入到$x$

$$[6.2656860351562501110223024625156540423631668090820312 5,$$
$$6.2656860351562498889776975374843459576368331909179687 5]$$

# 计算机运算

- **机器精度**
  - 浮点系统表示一个非零实数最大可能的相对误差，是浮点系统精度的刻画，表示为$\varepsilon_{mach}$
    - 截断舍入： $\varepsilon_{mach} = \beta^{1-p}$
    - 最近舍入： $\varepsilon_{mach} = \frac{1}{2}\beta^{1-p}$

$$o < UFL \ll \varepsilon_{mach} \ll OFL$$

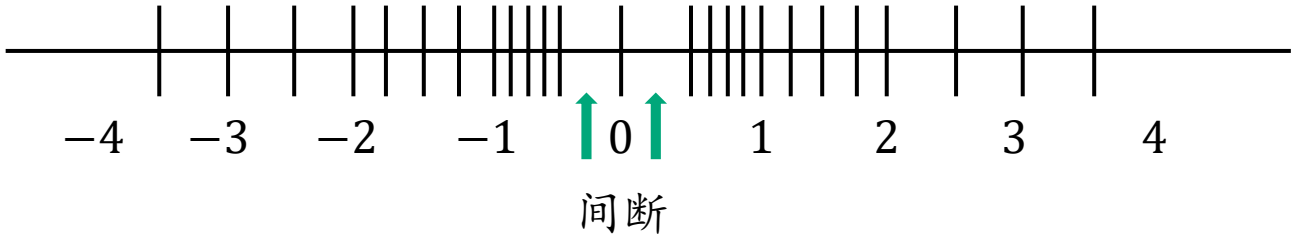
$$(\beta^{L}) \qquad (\beta^{U+1}(1-\beta^{-p}))$$

  - 机器精度和下溢限定义不同， $\varepsilon_{mach}$由尾数域的位数决定
  - UFL由指数域的位数决定，一般的有：

| | $UFL$ | $\varepsilon_{mach}$ （最近舍入） | $OFL$ |
|---|---|---|---|
| 单精度 | $2^{-126} \approx 1.2 \times 10^{-38}$ | $2^{-23}/2 \approx 6.0 \times 10^{-8}$ | $\approx 3.4 \times 10^{38}$ |
| 双精度 | $2^{-1022} \approx 2.2 \times 10^{-308}$ | $2^{-52}/2 \approx 1.1 \times 10^{-16}$ | $\approx 1.8 \times 10^{308}$ |

# 计算机运算

- **次正规化与逐渐下溢**
  - 正规化浮点系统中，由于限制尾数的首位不为0，导致0周围出现间断




  - 次正规化在指数达到最小值的时候，允许尾数首位为0，从而进一步降低下溢限，扩展表示范围
  - 次正规化表示精度低于正规化表示，因为会出现小数部分有效数字减少
  - 体现了”逐步下溢”，但并不会降低$\varepsilon_{mach}$

# 计算机运算

- **特殊值**
  - IEEE浮点系统提供了两个表示特殊情况的值：
    - Inf：表示无穷，有限数除零的结果，e.g. 1/0
    - NaN：非数，没有定义或者不确定的操作产生，e.g. 0*Inf
  - 通过特殊的指数域来实现
  - 是否支持取决于编程语言、编译以及运行系统
  - 有时可以用来处理特殊情况，一般应避免使用

# 计算机运算

- **浮点运算**
  - 浮点数加减：首先使指数相匹配，然后将尾数进行加减：
    - 指数匹配中，尾数移位可能导致某些位移出尾数域
    - 极端情况，操作数值差别过大，较小操作数可能完全丢失
  - 浮点数相乘/相除：指数相加/相减，尾数相乘/相除
    - 但$p$位尾数相乘/相除的结果超出$p$位数字，必须舍入
  - 一般的，浮点数运算结果与对应实数运算结果不同
  - 当计算结果超出浮点系统范围时，也不能表示(上溢和下溢)，下溢可以当做零，但上溢往往是严重错误

# 计算机运算

$\beta = 10$，$p = 6$的浮点系统

3位有效数字

$x = 1.92403 \times 10^2$,　　$y = 6.35782 \times 10^{-1} = 0.00636 \times 10^2$

| | 实数系统 | 浮点数系统 |
|---|---|---|
| $x + y$ | $1.93038782 \times 10^2$ | $1.93039 \times 10^2$ |
| $x * y$ | $1.22326364146 \times 10^2$ | $1.22326 \times 10^2$ |
| $x/y$ | $3.026241699198 \ldots \times 10^2$ | $3.02624 \times 10^2$ |

# 计算机运算

- **浮点运算**
  - 浮点数集合对运算不封闭，运算过程必然引入舍入误差
  - 只要运算结果在浮点数范围内，IEEE标准浮点系统可以通过浮点算术运算产生正确舍入结果
  - 某些实数域上的运算定律在浮点数集合中不满足：
    - 设$\varepsilon$是略小于$\varepsilon_{mach}$的数，则下式**违背 "结合律"**

$$1 + (\varepsilon + \varepsilon) = 1 + 2\varepsilon > 1 = (1 + \varepsilon) + \varepsilon$$

    - $\beta = 10$，$p = 2$的浮点系统中，下式违背 "分配率"

$$(1.2 \times 10^{-1} + 9.9 \times 10^{-1}) \times 5 = 1.1 \times 5 = 5.5$$

$$1.2 \times 10^{-1} \times 5 + 9.9 \times 10^{-1} \times 5 = 6 \times 10^{-1} + 5.0 = 5.6$$

    - 可能通过改变运算的顺序改善误差!

# 计算机运算

- **调和级数求和 $\sum_{n=1}^{+\infty} {}^{1}/_{n}$**
  - $1+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{5}+\frac{1}{6}+\frac{1}{7}+\frac{1}{8}+\cdots > 1+\frac{1}{2}+\frac{1}{2}+\frac{1}{2}+\cdots \to \infty$
  - 调和级数在实数域**发散**
  - 浮点数系统中直接运算，**调和级数收敛**！
    - 调和级数求和结果上溢?
    - 级数的项$1/n$下溢? 远没到下溢，$1/2097152 \approx 4.77\times 10^{-7} \gg \mathrm{UFL}(10^{-38})$
    - $1/n$相对部分和$\sum_{k=1}^{n-1} {}^{1}/_{k}$来说过小：$1/n \cdot 1/(\sum_{k=1}^{n-1} {}^{1}/_{k}) < \boldsymbol{\varepsilon_{mach}}$，其后的加法即不再起作用，精度完全损失
    - 单精度IEEE浮点系统中，约发生在$n \approx 2097152$，此时的调和级数部分和约为15.40

# 计算机运算

- **抵消(cancellation)**
  - 符号相同且值相近的数相减，可能造成有效数字减少：
    - $\beta = 10$，$p = 6$的浮点系统
    - $1.92403 \times 10^2 - 1.92275 \times 10^2 = 1.28000 \times 10^{-1}$，结果精确，但仅有三位有效数字
  - 抵消可能造成严重信息丢失：
    - 设$\varepsilon$是略小于$\varepsilon_{mach}$的数
    - $(1+\varepsilon)-(1-\varepsilon) = 1-1 = 0$，丢失了真实结果$2\varepsilon$
  - 舍入丢失的位常是次要的数位，而抵消的位通常是主要的有效数位
  - **应避免相近的大数相减，以免舍入误差在结果中主导**

# 计算机运算

一元二次方程$ax^2+bx+c=0$求根公式：

$$x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

**几种处理方式**

- 如果系数过大/过小，$b^2$或4ac可能上溢/下溢；可以调整系数，方程两边除以绝对值最大的系数
- 改用如下公式可以为其中一个根的计算避免$-b$和$\sqrt{b^2-4ac}$之间的抵消：

$$x=\frac{2c}{-b\mp\sqrt{b^2-4ac}}$$

- 平方根内的抵消难以消除，即判别式相对于系数很小的时候，两个根非常接近，问题病态

# 计算机运算

- 系数取4位有效数字：

$$a = 6.413 \times 10^{-2}, b = -8.642 \times 10^{1}, c = 3.277 \times 10^{0}$$

- 方程精确解：$x_1 = 1347.53, x_2 = 0.0379205$
- 判别式的平方根 $\sqrt{b^2 - 4ac} = 8.641 \times 10^{1}$
- 原求根公式：

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$x_1 = 1.347 \times 10^{3}, \boldsymbol{x_2 = \frac{86.42-86.41}{2\times6.413\times10^{-2}} = 7.797 \times 10^{-2}}$$

$$x = \frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$$

- 新求根公式：$\boldsymbol{x_1 = \frac{2\times3.277}{86.42-86.41} = 655.4}$ $x_2 = 3.793 \times 10^{-2}$
- 应根据系数的值选择公式以避免抵消

# 计算机运算

- **数值计算的特点**
  - 计算过程中的误差不可避免
  - 误差的来源和传播可以被分析
  - 误差的传播更应被重视，选择稳定的算法可对其加以控制
  - 良态的问题，采用稳定的算法可以得到满意的解

数据与算法

# 谢　谢！

北京科技大学
计算机与通信工程学院
皇甫伟/杜　冰